Folha de consulta — Aula 10

Aula 10 — O inimigo que contorna obstáculos e o ataque

Jogos Digitais II · 28/10/2026 · Lab 3.54

Prof. Ezéfferth C. A. Fernandes

Na Aula 09 o inimigo andava em linha reta até você. Basta uma casa no meio do caminho para ele ficar empurrando a parede enquanto você descansa do outro lado. Hoje ele passa a calcular o caminho e dar a volta. Em seguida você ganha o que faltava: um jeito de se defender.

Esta folha repete a demonstração na mesma ordem. Os nomes de nó são os do exemplo (a vila); no seu jogo, troque pelo que você tiver, mas mantenha a mesma estrutura.

Você parte do jogo da tag `aula-09`. Quem escolheu o modo plataforma vai direto para a seção "Modo plataforma", no fim.

Checkpoints

Ao terminar a	O jogo faz	Se não fizer
Parte 1	o inimigo dá a volta na casa em vez de empurrar a parede	as camadas com colisão ficaram dentro da `Navegacao`?
Parte 2	Espaço acende um clarão em volta de você e quem estiver nele some	a `Mask` do `Ataque` enxerga a camada 3?

Parte 1 — O inimigo que contorna

1.1 O problema

Antes de mexer, rode o jogo e se esconda atrás de uma casa. O inimigo para do outro lado, andando contra a parede. É o `direction_to` da Aula 09: ele aponta a seta para onde você está e não sabe que existe uma casa no caminho.

Para dar a volta, ele precisa de duas coisas:

Peça	Nó da Godot	O que faz
O mapa do chão livre	`NavigationRegion2D`	guarda os polígonos por onde dá para andar, já descontando muro, casa e árvore
Quem calcula o caminho	`NavigationAgent2D`	recebe o destino e devolve o próximo ponto da rota

A rota sai do A*, um algoritmo de caminho mínimo que a Godot já traz pronto. O seu trabalho é desenhar o mapa e, a cada quadro, perguntar ao agente qual é o próximo ponto.

1.2 O mapa do chão livre

Na `vila.tscn`:

1. Botão direito na `Vila` → **Add Child Node** → `NavigationRegion2D`. Renomeie para `Navegacao`.

2. Arraste o `Navegacao` para logo abaixo da `Vila`, acima de tudo. Assim o cenário continua sendo desenhado antes do jogador e dos inimigos.
3. Arraste as camadas do TileMap (`chao` e `objetos` no exemplo) para dentro do `Navegacao`. A região só enxerga a colisão do que estiver dentro dela. A camada das copas, se você tem uma, pode ir junto, sem efeito nenhum.

```
Vila
+-- Navegacao
|   +-- chao
|   +-- objetos
+-- Jogador
+-- Nascedouros
+-- Horda
+-- Relogio
+-- HUD
```

4. Com o `Navegacao` selecionado, no Inspetor: **Navigation Polygon** → **New NavigationPolygon**.
5. Ainda com ele selecionado, aparece uma barra de ferramentas em cima da tela da cena, com o lápis (**Create Points**) já ativo. Clique nos quatro cantos do mapa, por fora do muro, e feche clicando de novo no primeiro ponto. O contorno é o limite de onde o inimigo pode andar; não precisa ser exato.
6. Na mesma barra, clique em **Bake NavigationPolygon**. O mapa se enche de polígonos azuis, com um buraco em volta de cada casa, árvore e trecho de muro.

Os buracos são maiores que as casas, de propósito. A folga vem do **Agent Radius**, um campo do próprio `NavigationPolygon`, que já vem em 10: o inimigo é um corpo, não um ponto, e a rota precisa deixar espaço para ele passar sem raspar.

> **Não diminua o Agent Radius.** A colisão do inimigo fica nos pés, 5 px abaixo do centro dele. Com a folga em 6, a rota passa colada nas casas, os pés raspam na quina e ele engancha. Testamos as duas: com 10, o inimigo chegou ao jogador 22 vezes em 24; com 6, chegou 14.

Mudou o mapa depois? Acrescentou uma casa, apagou uma árvore? Clique em **Bake** de novo. O mapa do chão livre não se atualiza sozinho.

1.3 O agente no inimigo

Abra `entities/inimigo.tscn` e dê ao `Inimigo` um filho `NavigationAgent2D` (botão direito → **Add Child Node**). No Inspetor do agente, em **Pathfinding**:

Campo	Valor	Por quê
Path Desired Distance	4	com o padrão 20, ele dá o ponto como alcançado cedo demais, corta a quina e engancha
Target Desired Distance	4	com o padrão 10, ele para antes de encostar em você e nunca tira vida

Salve com Ctrl+S, com a aba do `inimigo.tscn` na frente.

1.4 O script do inimigo

`scripts/inimigo.gd` inteiro, substituindo o da Aula 09:

```
extends CharacterBody2D

@export var velocidade: float = 40.0

@onready var agente: NavigationAgent2D = $NavigationAgent2D


func _physics_process(_delta: float) -> void:
    var jogador := get_tree().get_first_node_in_group("player") as Node2D
    if jogador == null:
        return

    # O agente calcula o caminho; o inimigo só anda até o próximo ponto dele.
    agente.target_position = jogador.global_position
    var proximo := agente.get_next_path_position()
    velocity = global_position.direction_to(proximo) * velocidade
    move_and_slide()
```

Compare com o da Aula 09. Entraram o `@onready` do agente e duas linhas no `_physics_process`: o destino passa a ser o jogador, e a seta aponta para o próximo ponto da rota, não mais para ele. O resto, `velocidade`, `move_and_slide` e o grupo `player`, ficou igual.

Teste: ligue **Debug** → **Visible Navigation** e **Debug** → **Visible Collision Shapes**, rode e se esconda atrás de uma casa. O inimigo contorna pela borda azul.

> **Isso custa.** A cada quadro, cada inimigo recalcula a rota inteira. Com dez inimigos não se nota; com a horda cheia, começa a contar. Na Aula 11, de otimização, é uma das primeiras coisas que vamos medir.

Parte 2 — O ataque

Até aqui você só fugia. O ataque é um círculo em volta do jogador: apertou Espaço, todo inimigo que estiver dentro dele leva o golpe.

2.1 A tecla

Project → **Project Settings** → **Input Map**. Digite `atacar` em **Add New Action** e clique em **Add**. Na linha nova, clique no +, aperte Espaço e confirme com **OK**. A linha passa a mostrar `Space`.

2.2 O círculo do golpe

Em `entities/jogador.tscn`, filho do `Jogador`: uma **`Area2D`** chamada `Ataque`. **Layer** nenhuma, **Mask** só na 3, como a `Hurtbox`. Dentro dela:

Filho	Como
`CollisionShape2D`	`CircleShape2D` de raio 20, posição (0, 2)
`ColorRect` chamado `Golpe`	**Color** branco com o **A** (transparência) em 90; **Layout** → **Transform**: **Size** (40, 40) e **Position** (-20, -18); **Visible** desmarcado

O `Golpe` é só o clarão: mostra na tela o tamanho do golpe. Sem ele, você aperta Espaço e não sabe se errou a tecla ou o alcance.

```
Jogador
+-- AnimatedSprite2D
+-- CollisionShape2D
```

```
+-- Camera2D
+-- Hurtbox
|   +-- CollisionShape2D
+-- Ataque
    +-- CollisionShape2D
    +-- Golpe
```

2.3 O script do jogador

Em `scripts/jogador.gd`, quatro acréscimos. O resto do arquivo não muda, e é melhor não colar o arquivo inteiro por cima do seu: se as suas animações ou os seus nós têm outros nomes (`right` em vez de `direita`, por exemplo), o arquivo da folha quebra o que já funcionava.

1. Abaixo de `var invencivel := false`:

```
var pode_atacar := true
```

2. Abaixo do `@onready` da `hurtbox`:

```
@onready var ataque: Area2D = $Ataque
@onready var golpe: ColorRect = $Ataque/Golpe
```

3. No fim do `_process`, depois do `levar_dano()`, com um Tab de recuo:

```
    if pode_atacar and Input.is_action_just_pressed("atacar"):
        atacar()
```

4. No fim do arquivo, a função nova:

```
func atacar() -> void:
    # Acerta todo inimigo que estiver dentro do círculo neste quadro.
    for inimigo in ataque.get_overlapping_bodies():
        inimigo.levar_golpe()

    # O clarão dura 0,1 s; o próximo golpe só sai 0,4 s depois deste.
    pode_atacar = false
    golpe.visible = true
    await get_tree().create_timer(0.1).timeout
    golpe.visible = false
    await get_tree().create_timer(0.3).timeout
    pode_atacar = true
```

O `_process` fica assim:

```
func _process(_delta: float) -> void:
    # Tem inimigo encostando em mim?
    if not invencivel and hurtbox.has_overlapping_bodies():
        levar_dano()

    if pode_atacar and Input.is_action_just_pressed("atacar"):
        atacar()
```

O `atacar()` é parente do `levar_dano()`: faz o efeito, trava, espera com `await` e destrava. Lá a trava era o `invencivel`; aqui é o `pode_atacar`. Sem ela, quem martela o Espaço limpa a tela, e a horda deixa de ser ameaça.

O `get_overlapping_bodies()` é o irmão do `has_overlapping_bodies()` da Aula 09. Em vez de responder sim ou não, devolve a lista de quem está dentro. O `for` passa por cada um.

2.4 O inimigo que morre

No `inimigo.gd`, abaixo da `velocidade`, e a função no fim do arquivo:

```
@export var vida: int = 1
```

```
func levar_golpe() -> void:
    vida -= 1
    if vida <= 0:
        get_tree().current_scene.inimigo_morreu(self)
        queue_free()
```

`queue_free()` tira o inimigo do jogo no fim do quadro. Antes disso ele avisa a vila: é ela quem conta os abates, e algumas fichas do fim da folha usam essa contagem para decidir a vitória.

2.5 A vila conta os abates

No `vila.gd`, abaixo da `cena_inimigo`:

```
var abates := 0
```

A função, abaixo do `_process`, fora de qualquer outra função (a linha do `func` sem recuo):

```
func inimigo_morreu(_inimigo: Node2D) -> void:
    abates += 1
```

E, no `_process`, troque a linha da vida para mostrar os dois números:

```
    texto_vida.text = "Vida: %d   Abates: %d" % [jogador.vida, abates]
```

Teste: deixe a horda chegar e aperte Espaço com os inimigos encostados. O clarão aparece, eles somem e o placar sobe.

A entrega de hoje

A tag `aula-10` com as Partes 1 e 2 funcionando: o inimigo contornando os obstáculos do seu cenário e o ataque com o placar de abates.

Terminou? A seção "O objetivo do seu tipo de jogo", no fim da folha, é o passo seguinte. Ela entra no projeto em cima da tag de hoje, e é bom ir para a Aula 11 com ela pronta.

Modo plataforma

No jogo de plataforma o inimigo não precisa de mapa de navegação: ele só anda para a esquerda ou para a direita. O problema dele é outro: ele persegue você e cai no primeiro buraco, como ficou anotado na Aula 06. Ataque você já tem desde lá.

A saída é um raio que olha o chão à frente dos pés. Na cena do seu zumbi, filho dele: um **`RayCast2D`** chamado `OlhaChao`. Em **Transform** → **Position**, ponha o raio logo depois da borda da colisão, no lado para onde o sprite olha: no zumbi grande da Aula 06, cuja colisão tem 20 px de raio, (24, 0). Nos outros zumbis, meça pela colisão de cada um. O **Target Position** fica no padrão, (0, 50), apontando para baixo.

O raio tem que passar dos pés. No zumbi grande, os pés ficam 32 px abaixo do centro; um raio de 24 nunca toca o chão, o zumbi acha que está sempre na beirada e não sai do lugar. Confira no editor: a seta do raio precisa atravessar a linha do chão.

No `zumbi.gd`, dentro da `olhar_para()`, abaixo da linha da `area_ataque`, para o raio virar junto:

```
    $OlhaChao.position.x = abs($OlhaChao.position.x) * lado
```

E no `chase_state()`, troque as duas últimas linhas (`velocity.x = ...` e `olhar_para(lado)`) por:

```
    olhar_para(lado)

    # Sem chão à frente: para na beirada em vez de cair.
    if is_on_floor() and not $OlhaChao.is_colliding():
        velocity.x = 0
        return

    velocity.x = lado * velocidade
```

Teste: atraia o zumbi até a beira de um buraco. Ele para na beirada e espera; você pulou para o outro lado e está a salvo. Volte para o lado dele e ele torna a perseguir, porque o raio vira junto com o sprite.

O exemplo pronto

O resultado das Partes 1 e 2 está no repositório da disciplina, na branch `aula-10`: a vila da Aula 09 com a malha de navegação, o inimigo com o agente e o ataque.

Sem git: abra `github.com/ezefferth/construcao-jogo-godot/tree/aula-10`, use **Code** → **Download ZIP** e importe na Godot numa pasta à parte. Com git, também numa pasta à parte:

```
git clone -b aula-10 https://github.com/ezefferth/construcao-jogo-godot.git
↪  exemplo-aula-10
```

O exemplo é para consulta: abra lado a lado com o seu e compare. Não copie a pasta por cima do seu projeto.

Commit e tag

DEVLOG com a entrada de hoje (o que fez, o que travou, como resolveu) e:

```
git add .
git commit -m "aula-10: inimigo contorna obstaculos e ataque do jogador"
git tag aula-10
git push origin main --tags
```

Se sobrar tempo

1. **Inimigo que aguenta mais.** No Inspetor do `Inimigo`, `vida` 3. Para você ver que o golpe pegou, ele fica meio transparente por 0,1 s quando apanha e não morre. No `levar_golpe()`, ponha um `return` logo abaixo do `queue_free()` e, no fim da função, com um Tab de recuo:

```
    modulate.a = 0.4
    await get_tree().create_timer(0.1).timeout
    modulate.a = 1.0
```

2. **Rota mais barata.** Hoje cada inimigo recalcula o caminho 60 vezes por segundo. Filho do `Inimigo`: um `Timer` com **Wait Time** 0,2 e **Autostart** ligado, com o `timeout` conectado ao

próprio `Inimigo`. As linhas que buscam o `jogador` e a do `agente.target_position` saem do `_physics_process` e vão para a função do sinal:

```
func _on_timer_timeout() -> void:
    var jogador := get_tree().get_first_node_in_group("player") as Node2D
    if jogador == null:
        return
    agente.target_position = jogador.global_position
```

No `_physics_process` ficam só as três linhas do `proximo`, da `velocity` e do `move_and_slide()`. O inimigo anda a cada quadro, mas recalcula o caminho cinco vezes por segundo. No teste, ele chegou ao jogador nas mesmas 22 vezes em 24.

3. **Inimigo com outra velocidade.** Um `inimigo_rapido.tscn` duplicado, com `velocidade` 60. Ele alcança você nas retas, e nas curvas a rota é a mesma.

As armadilhas desta aula

- **O inimigo continua empurrando a parede.** As camadas do TileMap não estão dentro do `Navegacao`, ou faltou clicar em **Bake**. Com **Visible Navigation** ligado, o mapa deve aparecer azul, com buracos nas casas.
- **O mapa azul cobre as casas, sem buraco.** As camadas ficaram fora do `Navegacao` na hora do Bake. Arraste para dentro e faça o Bake de novo.
- **Nenhum azul aparece.** O contorno não foi fechado: o último clique tem que ser no primeiro ponto.
- **O inimigo engancha nas quinas.** O **Agent Radius** foi diminuído, ou as duas distâncias do agente ficaram no padrão, 20 e 10.
- **O inimigo para perto de você e não tira vida. Target Desired Distance** ficou em 10.
- **"Invalid assignment of property or key 'target_position' ... on a base object of type 'null instance'".** O `NavigationAgent2D` não foi salvo dentro do `inimigo.tscn`, ou ganhou outro nome.
- **Mudou o mapa e o inimigo atravessa a casa nova.** Faltou repetir o **Bake**.
- **Espaço não faz nada.** A ação `atacar` não foi criada, ou foi criada com outro nome.
- **O clarão aparece mas ninguém morre.** A **Mask** do `Ataque` não tem a camada 3, ou o `CollisionShape2D` ficou fora do `Ataque` na árvore.
- **"Nonexistent function 'levar_golpe'".** A função não entrou no `inimigo.gd`, ou foi escrita com outro nome.
- **"Nonexistent function 'inimigo_morreu'".** A função não entrou no `vila.gd`.
- **O clarão fica torto.** Os números de **Position** e **Size** foram parar no `Ataque`, e não no `Golpe`.
- **Plataforma: o zumbi não sai do lugar.** O `OlhaChao` é curto demais e não alcança o chão, ou ficou dentro do corpo do zumbi. Com **Visible Collision Shapes**, a seta tem que atravessar o chão, à frente dele.
- **"Mixed use of tabs and spaces".** O código colado do PDF veio com espaços no lugar de Tab. Apague o recuo das linhas coladas e refaça com a tecla Tab.

O que você construiu hoje

Um inimigo que lê o cenário e um jogador que reage. O mapa do chão livre serve para qualquer personagem que precise andar sozinho: um aliado, um animal, um chefe. Na próxima aula a arte do seu tema entra no lugar do boneco vermelho, e a horda cheia passa a pesar no desempenho.

O objetivo do seu tipo de jogo

Estas fichas partem do jogo da tag `aula-10`. Cada uma diz o que o `Relogio` faz no seu tipo, as peças novas e o código. Os trechos se somam ao que você já tem; quando uma função substitui outra, a ficha avisa. O ataque e o `inimigo_morreu()` da Parte 2 já estão no jogo: Abate e Chefe só os usam.

Tipo	O `Relogio`	Peça nova
Sobrevivência	180 s e vence	nenhuma: só o tempo
Abate	desligado	a vitória pelo contador
Chefe	60 s e o chefe surge	um inimigo grande
Coleta	desligado	item que nasce em pontos sorteados
Defesa	180 s e vence	objeto que os inimigos atacam
Fuga	60 s e a saída abre	saída escondida
Território	desligado	zona que conta tempo e muda de lugar
Resgate	desligado	aliados que seguem em fila

"Desligado" quer dizer: no `Relogio`, desmarque **Autostart**. A linha do `texto_tempo` passa a mostrar 0; troque-a pelo seu contador, como cada ficha mostra.

Sobrevivência — aguentar 3 minutos

`Relogio` em 180 s. O resto é design: o `0.95` da horda, a velocidade do inimigo e a recarga do ataque decidem se três minutos são fáceis ou impossíveis.

Abate — derrotar 50 inimigos

No `vila.gd`, substitua a `inimigo_morreu` por esta, que acrescenta a condição de vitória:

```
func inimigo_morreu(_inimigo: Node2D) -> void:
    abates += 1
    if abates >= 50:
        vencer()
```

`Relogio` desligado. No `_process`, troque a linha do `texto_tempo` por `texto_tempo.text = "Faltam:  %d" % (50 - abates)`.

Chefe — resistir até o chefe surgir e derrotá-lo

O chefe é o mesmo inimigo, com mais vida e o desenho maior. No `vila.gd`, substitua a função `_on_relogio_timeout` e a `inimigo_morreu` por estas:

```
var chefe: Node2D = null


func _on_relogio_timeout() -> void:
    chefe = cena_inimigo.instantiate()
    chefe.vida = 10
    chefe.get_node("Sprite2D").scale = Vector2(2, 2)
    chefe.global_position = $Nascedouros/Cima.global_position
    add_child(chefe)


func inimigo_morreu(inimigo: Node2D) -> void:
    abates += 1
```

```
    if inimigo == chefe:
        vencer()
```

Só o desenho dobra, não o corpo inteiro. Com `chefe.scale`, a colisão também dobra, fica maior que a folga do mapa de navegação, e o chefe engancha em toda quina: no nosso teste ele chegou ao jogador 9 vezes em 24. `Relogio` em 60 s: é a contagem até o chefe aparecer.

Coleta — juntar 20 itens

O item é uma cena nova: `Area2D` chamada `Item` (Layer nenhuma, Mask só na 2), com um `Sprite2D` (o `tiny-town.png` com **Region** ligada num cogumelo, `Rect2(80, 32, 16, 16)`, ou qualquer arte) e um `CircleShape2D` de raio 6. Salve em `entities/item.tscn` e conecte o `body_entered` da `Item` a ela mesma. Script `scripts/item.gd`:

```
extends Area2D


func _on_body_entered(_body: Node2D) -> void:
    get_tree().current_scene.coletou()
    queue_free()
```

Na vila: um `Node2D` chamado `Pontos` com uns oito `Marker2D` espalhados em chão livre, e um `Timer` chamado `Itens` (Wait Time 3, Autostart), com o `timeout` conectado à `Vila`. No `vila.gd`, abaixo da `cena_inimigo`, `@export var cena_item: PackedScene` (arraste o `item.tscn` no Inspetor) e, no fim:

```
var coletados := 0


func _on_itens_timeout() -> void:
    var item := cena_item.instantiate()
    item.global_position = $Pontos.get_children().pick_random().global_position
    add_child(item)


func coletou() -> void:
    coletados += 1
    if coletados >= 20:
        vencer()
```

`Relogio` desligado; no placar, `"Itens: %d / 20" % coletados`.

Defesa — proteger um objeto por 3 minutos

O objeto é uma cena nova: `Area2D` chamada `Objeto`, no grupo `objeto` (Layer nenhuma, Mask só na 3), com um `Sprite2D` (o poço, `Rect2(128, 128, 16, 16)`) e um `CircleShape2D` de raio 10. Conecte o `body_entered` a ela mesma. Script `scripts/objeto.gd`:

```
extends Area2D

var toques := 0


func _on_body_entered(inimigo: Node2D) -> void:
    inimigo.queue_free()
    toques += 1
    if toques >= 5:
        get_tree().current_scene.perder()
```

Ponha um `Objeto` na vila, em chão livre. Agora o inimigo persegue o objeto, e não você.

No `inimigo.gd`, abaixo da `velocidade`, `@export var grupo_alvo: String = "player"`, e troque `get_first_node_in_group("player")` por `get_first_node_in_group(grupo_alvo)`. No `vila.gd`, na `_on_horda_timeout`, antes do `add_child`:

```
    inimigo.grupo_alvo = "objeto"
```

`Relogio` em 180 s. O ataque da Parte 2 é a sua defesa: você corre até os inimigos antes que cheguem ao objeto.

Fuga — sobreviver até a saída abrir, e alcançá-la

Na vila: uma `Area2D` chamada `Saida` (Layer nenhuma, Mask só na 2), longe do jogador, com um `Sprite2D` e um `CircleShape2D` de raio 10. No Inspetor dela, desmarque **Visible** e **Monitoring**: a saída começa invisível e sem perceber ninguém. Conecte o `body_entered` da `Saida` à `Vila`. No `vila.gd`, substitua a `_on_relogio_timeout` por estas duas:

```
func _on_relogio_timeout() -> void:
    $Saida.visible = true
    $Saida.monitoring = true


func _on_saida_body_entered(_body: Node2D) -> void:
    vencer()
```

`Relogio` em 60 s: é a contagem até a saída abrir.

Território — somar 60 segundos dentro da zona

Na vila: uma `Area2D` chamada `Zona` (Layer nenhuma, Mask só na 2), com um `ColorRect` amarelo transparente de 48 × 48 centrado nela e um `RectangleShape2D` do mesmo tamanho. Um `Node2D` chamado `Pontos` com `Marker2D` em chão livre, e um `Timer` chamado `MudaZona` (Wait Time 10, Autostart), com o `timeout` conectado à `Vila`. No `vila.gd`, no fim:

```
var tempo_na_zona := 0.0


func _physics_process(delta: float) -> void:
    if $Zona.has_overlapping_bodies():
        tempo_na_zona += delta
        if tempo_na_zona >= 60:
            vencer()


func _on_muda_zona_timeout() -> void:
    $Zona.global_position = $Pontos.get_children().pick_random().global_position
```

`Relogio` desligado; no placar, `"Zona: %d / 60" % tempo_na_zona`.

Resgate — encontrar 5 aliados, que seguem em fila

O aliado é uma cena nova: `Area2D` chamada `Aliado` (Layer nenhuma, Mask na 2 e na 3), com um `Sprite2D` (o personagem com **Modulate** azul) e um `CircleShape2D` de raio 6. Conecte o `body_entered` a ela mesma. Script `scripts/aliado.gd`:

```
extends Area2D

var lider: Node2D = null
```

```
func _physics_process(delta: float) -> void:
    if lider and global_position.distance_to(lider.global_position) > 14:
        global_position = global_position.move_toward(lider.global_position, 70 * delta)


func _on_body_entered(body: Node2D) -> void:
    if body.is_in_group("player") and lider == null:
        get_tree().current_scene.resgatou(self)
    elif body.is_in_group("inimigo") and lider != null:
        get_tree().current_scene.perder()
```

Espalhe cinco `Aliado` pela vila, em chão livre. No `vila.gd`, no fim:

```
var fila: Array[Node2D] = []


func resgatou(aliado: Node2D) -> void:
    if fila.is_empty():
        aliado.lider = jogador
    else:
        aliado.lider = fila.back()
    fila.append(aliado)
    if fila.size() >= 5:
        vencer()
```

O primeiro aliado segue você; cada um dos seguintes segue o anterior. É a cobrinha: `lider` é quem vai na frente. `Relogio` desligado; no placar, `"Aliados:  %d / 5" % fila.size()`.